

MNIST 手写识别项目三

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

class DigitClassifier(nn.Module):
    def __init__(self):
        super(DigitClassifier, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = x.view(-1, 28 * 28)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

full_dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
train_size = 50000
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# 初始化
model = DigitClassifier()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 早停机制
best_val_loss = float('inf')
```

```

patience = 3
patience_counter = 0
epochs = 30
for epoch in range(epochs):
    model.train()
    train_loss = 0.0
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in val_loader:
            output = model(data)
            loss = criterion(output, target)
            val_loss += loss.item()
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()
            total += target.size(0)
    avg_val_loss = val_loss / len(val_loader)
    val_acc = 100 * correct / total
    print(f'Epoch {epoch + 1}: Train Loss: {train_loss / len(train_loader):.4f},
,
        f'Val Loss: {avg_val_loss:.4f}, Val Acc: {val_acc:.2f}%')
# 早停判断
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    patience_counter = 0
    torch.save(model.state_dict(), 'best_model.pth')
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f'Early stopping at epoch {epoch + 1}')

```

```
        break

# 加载最佳模型并在测试集上评估
model.load_state_dict(torch.load('best_model.pth'))
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()
        total += target.size(0)
print(f'Test Accuracy (best model): {100 * correct / total:.2f}%')
```